

E-Signature Platform: Data Model Specification

Three-party signing platform (company / agency / supplier) with multi-entity branding, snapshot-and-bake document immutability, evidence upload for one role, and a tamper-evident audit trail.

Recommended stack: Cloud SQL (Postgres), Google Cloud Storage for binary artifacts, transactional email provider with per-entity verified domains. Relational integrity matters here because the audit story depends on it; do not use Firestore for this.

Core invariants (enforce these everywhere)

1. Once an envelope is baked, its document, party details, and field definitions are immutable. No exceptions. Changes mean void and re-issue.
 2. Every signature event records the SHA-256 of the document as it existed at that moment. It must match the baked hash or the signature is invalid.
 3. The events table is append-only. No UPDATE, no DELETE, ever. Enforce at the DB role level, not just in application code.
 4. The entity profile is the single source of truth for both branding AND legal party details. Nothing else supplies company names, numbers, or addresses to a document.
 5. Evidence files are encrypted at rest, access-logged, and carry a retention expiry.
-

1. entities

One row per sending/owning company.

```
entities (  
  id                uuid PK,  
  slug              text UNIQUE,           -- 'acme', used in from-addresses and URL  
  legal_name        text NOT NULL,         -- exact registered name, appears in cont  
  company_number    text NOT NULL,  
  vat_number        text,  
  registered_office text NOT NULL,  
  display_name      text NOT NULL,         -- trading/brand name for UI and email
```

```

brand          jsonb NOT NULL,          -- { primary, secondary, logo_asset_id, f
sending_domain text NOT NULL,          -- must be DNS-verified before use
from_address   text NOT NULL,          -- e.g. documents@acme.co.uk
reply_to       text,
email_signature_html text NOT NULL,    -- the entity's defined signature block
email_signature_text text NOT NULL,    -- plain-text equivalent
domain_verified boolean DEFAULT false,
is_active      boolean DEFAULT true,
created_at     timestampz
)

```

Rule: an entity with `domain_verified = false` cannot be selected at envelope creation. Fail loudly, not silently.

2. entity_assets

Cover library and brand assets, scoped per entity.

```

entity_assets (
  id          uuid PK,
  entity_id   uuid FK -> entities,
  kind        text CHECK (kind IN ('front_cover', 'back_cover', 'logo', 'email_heade
  name        text,          -- 'Formal Navy', 'Casual 2026'
  storage_ref text NOT NULL,  -- GCS object path
  mime        text,
  page_size   text,          -- 'A4' etc, validate covers match doc si
  is_active   boolean DEFAULT true,
  created_at  timestampz
)

```

3. email_templates

Per-entity, per-message-type email content. Every email the system sends is one of these types; there are no unbranded system emails.

```

email_templates (
  id          uuid PK,
  entity_id   uuid FK -> entities,
  template_type text CHECK (template_type IN
                                ('invitation', 'resend', 'reminder', 'turn_notification',
                                'declined_notice', 'voided_notice', 'completion')),
  subject     text NOT NULL,          -- supports merge variables
  body_html   text NOT NULL,

```

```

body_text      text NOT NULL,          -- plain-text part, required for delivera
is_active      boolean DEFAULT true,
updated_by     uuid FK -> users,
updated_at     timestamptz,
UNIQUE (entity_id, template_type)
)

```

Rules:

- The signature is NOT part of the body template. It is appended at send time from entities.email_signature_html/text. One signature edit propagates to all six template types instantly and nothing can drift.
- Merge variables available in subject and body: {{signer_name}}, {{signer_company}}, {{entity_display_name}}, {{envelope_title}}, {{signing_link}}, {{expiry_date}}, {{role_label}}, {{declined_by}}, {{decline_reason}}. Rendering fails loudly on an unknown variable; a template referencing {{signng_link}} must not save.
- {{signing_link}} is mandatory in invitation, resend and turn_notification templates: validated at template save, not discovered at send time.
- New entities are seeded with sensible defaults for all six types, so a missing template can never block an envelope. Editing is optional, sending is not.
- Every send logs an event (invitation_sent etc.) whose metadata records the template id and a hash of the rendered content, so “what exactly did we send them” is answerable later.

4. templates and template_versions

Lifecycle rule for all entity-scoped resources (assets, email templates)

Anything ever referenced by an envelope is permanent. is_active = false removes it from selection UIs; hard delete is only permitted for resources with zero envelope references. Enforce with FK constraints plus an application-level guard on the delete path. This is what keeps “show me envelope #300 exactly as it went out” answerable in three years.

Admin console (build scope, not schema)

The management surface implied by all of the above, per entity:

- Entity profile editor: legal details, brand tokens, signature (with preview)
- Sending domain setup: DNS records shown, verification check, status badge

- Asset library: upload covers/logos with format validation, preview at A4, activate/deactivate
- Email template editor: per-type editing with merge-variable insertion, live preview using sample data, plain-text auto-draft from HTML, send-test-email
- Template (contract) management: link, fetch preview, version history

Staff access to each entity's console is governed by `user_entity_access`.

The template holds the live contract link. A version is created lazily: at bake time the link is fetched, and if the content hash differs from the latest version, a new version row is written. Envelopes always reference a version, never the template.

```
templates (
  id          uuid PK,
  entity_id   uuid FK -> entities NULL,  -- NULL = shared across entities
  name        text NOT NULL,
  source_url   text NOT NULL,              -- the per-client contract link
  default_front_cover uuid FK -> entity_assets NULL,
  default_back_cover  uuid FK -> entity_assets NULL,
  is_active    boolean DEFAULT true,
  created_at   timestampz
)

template_versions (
  id          uuid PK,
  template_id uuid FK -> templates,
  version_no   int,              -- monotonic per template
  content_hash text NOT NULL,     -- SHA-256 of fetched source
  storage_ref  text NOT NULL,     -- the fetched, rendered PDF snapshot
  fetched_at   timestampz,
  UNIQUE (template_id, content_hash)
)
```

This answers “which contract wording did envelope #4571 use” permanently.

5. template_roles and template_fields

Role definitions live on the template so envelope creation is fast and consistent.

```
template_roles (
  id          uuid PK,
  template_id uuid FK -> templates,
  role_key     text CHECK (role_key IN ('company','agency','supplier')),
```

```

    label            text,
    signing_order    int NOT NULL,          -- 1, 2, 3; equal values = parallel
    evidence_required boolean DEFAULT false, -- true when requirements exist below
    UNIQUE (template_id, role_key)
)

-- Typed evidence checklist per role. For the client (company role) this is:
-- proof of company address, proof of directors, VAT certificate,
-- certificate of incorporation. Attached to the role, so it is configurable
-- per template rather than hard-coded.
evidence_requirements (
    id            uuid PK,
    role_id       uuid FK -> template_roles,
    requirement_key text NOT NULL,          -- 'proof_of_address', 'proof_of_director
                                           -- 'vat_certificate', 'incorporation_cert

    label        text NOT NULL,
    description   text,                    -- shown to the uploader: what qualifies
    accepted_mimes text[],                 -- e.g. {application/pdf, image/jpeg}
    max_size_bytes bigint,
    verify_method text CHECK (verify_method IN
                              ('upload', 'companies_house_api', 'hmrc_vat_api')),
    is_required   boolean DEFAULT true,
    UNIQUE (role_id, requirement_key)
)

template_fields (
    id            uuid PK,
    template_id   uuid FK -> templates,
    role_id       uuid FK -> template_roles, -- ONLY this role may complete it
    field_key     text,                      -- 'po_number', 'insurance_ref'
    label        text,
    field_type    text CHECK (field_type IN
                              ('text', 'date', 'checkbox', 'signature', 'initials', 'attachment')),
    required      boolean DEFAULT true,
    page         int,                        -- position in the baked document
    x            numeric, y numeric, w numeric, h numeric, -- in points
    validation    jsonb                     -- regex, max length, date bounds
)

```

Field scoping is a hard rule: the API rejects any field value submitted by a party whose role does not own that field. Do not rely on the UI hiding fields.

6. envelopes

```

envelopes (

```


Who fills each role on a specific envelope. This is also where access tokens live.

```
envelope_parties (
  id          uuid PK,
  envelope_id  uuid FK -> envelopes,
  role_id      uuid FK -> template_roles,
  company_name text NOT NULL,          -- the party's org as named in the contra
  signer_name  text NOT NULL,
  signer_email text NOT NULL,
  status       text CHECK (status IN
                          ('pending','notified','viewed','in_progress',
                          'completed','declined')),
  order_index  int NOT NULL,          -- copied from role signing_order at bake
  declined_reason text,
  completed_at timestamptz
)

party_access_tokens (
  id          uuid PK,
  party_id     uuid FK -> envelope_parties,
  token_hash   text NOT NULL,          -- store the hash, never the raw token
  expires_at   timestamptz NOT NULL,
  used_at      timestamptz,          -- single-use for the signing action itse
  revoked_at   timestamptz,
  created_at   timestamptz
)
```

Token rules: raw token appears only in the emailed link; DB stores its hash. Reissuing (resend invitation) revokes prior tokens for that party. Voiding the envelope revokes all tokens. Viewing can reuse a token until expiry; the final sign action burns it.

8. field_values

```
field_values (
  id          uuid PK,
  envelope_id  uuid FK -> envelopes,
  field_id     uuid FK -> template_fields,
  party_id     uuid FK -> envelope_parties, -- must match field's role owner
  value        text,          -- or storage_ref for attachment type
  value_hash   text,          -- SHA-256, included in completion certif
  captured_at  timestamptz,
  captured_ip  inet,
  UNIQUE (envelope_id, field_id)
)
```

9. signatures

Separate from field_values because signatures carry heavier evidential metadata.

```
signatures (  
  id                uuid PK,  
  party_id          uuid FK -> envelope_parties,  
  envelope_id       uuid FK -> envelopes,  
  method            text CHECK (method IN ('drawn','typed','uploaded')),  
  signature_asset_ref text,                -- rendered signature image in GCS  
  document_hash     text NOT NULL,        -- MUST equal envelopes.baked_hash  
  consent_text      text NOT NULL,        -- exact wording they agreed to  
  consent_given_at  timestampz NOT NULL,  
  signed_at         timestampz NOT NULL,  
  ip                inet NOT NULL,  
  user_agent        text NOT NULL  
)
```

The consent_text row is the “I agree to sign electronically” wording as displayed at that moment. Store it verbatim per signature; do not reference a mutable config.

10. evidence_files

The identity/documentary evidence the client role must attach. Each upload satisfies exactly one requirement; the signing gate checks the requirement checklist, not a file count.

```
evidence_files (  
  id                uuid PK,  
  party_id          uuid FK -> envelope_parties,  
  envelope_id       uuid FK -> envelopes,  
  requirement_id     uuid FK -> evidence_requirements NOT NULL,  
  review_status      text CHECK (review_status IN  
                                ('uploaded','accepted','rejected'))  
                                DEFAULT 'uploaded',  
  rejected_reason    text,                -- surfaced to the party on re-request  
  original_name      text,  
  mime              text,  
  size_bytes         bigint,  
  storage_ref        text NOT NULL,        -- GCS, bucket with CMEK encryption  
  file_hash          text NOT NULL,  
  uploaded_at        timestampz,  
  uploaded_ip        inet,  
  retention_expires_at timestampz NOT NULL, -- GDPR: set at upload, job deletes  
  deleted_at         timestampz
```


)

Access to evidence files is itself an audited event. Decide retention now (e.g. 6 years to match contract limitation periods, or shorter if the evidence is only needed pre-completion) and enforce it with a scheduled deletion job.

For requirements with `verify_method = companies_house_api` or `hmrc_vat_api`, no file is uploaded; the system performs a live register lookup and stores the result:

```
verification_results (  
  id          uuid PK,  
  party_id    uuid FK -> envelope_parties,  
  requirement_id uuid FK -> evidence_requirements,  
  provider     text,                -- 'companies_house', 'hmrc_vat'  
  query_value  text,                -- company number or VAT number submitted  
  result       text CHECK (result IN ('verified','not_found','mismatch')),  
  response_snapshot jsonb,          -- raw API response, retained as evidence  
  checked_at   timestampz  
)
```

A stored Companies House response is stronger evidence than an uploaded PDF of an incorporation certificate, and it costs nothing. Uploads remain the fallback for documents with no public register (proof of address).

11. documents

Every binary artifact the system produces, hashed.

```
documents (  
  id          uuid PK,  
  envelope_id uuid FK -> envelopes,  
  kind        text CHECK (kind IN  
                          ('snapshot','baked','completed','certificate')),  
  storage_ref text NOT NULL,  
  sha256      text NOT NULL,  
  size_bytes  bigint,  
  created_at  timestampz  
)
```

- snapshot: the fetched contract from the link (also referenced by `template_versions`)
- baked: covers + contract + field layer, pre-signing. The thing everyone signs.
- completed: baked doc with all field values and signature images applied

- certificate: appended audit certificate (signers, timestamps, IPs, hashes)

Distribution rule: on completion, each party receives completed + certificate. Appending the certificate AFTER signing is fine and standard; altering the signed pages is not. The certificate references the baked_hash so the two are bound.

12. events (append-only audit log)

```
events (
  id          bigserial PK,
  envelope_id uuid FK -> envelopes NULL,
  party_id    uuid FK -> envelope_parties NULL,
  actor       text,                                -- 'party', 'system', 'staff:<user_id>'
  event_type  text NOT NULL,
  metadata    jsonb,
  ip          inet,
  user_agent  text,
  created_at  timestampz NOT NULL DEFAULT now(),
  prev_hash   text NOT NULL,                        -- hash chain: SHA-256 of previous event
  row_hash    text NOT NULL                         -- SHA-256 of this row's canonical form
)
```

Event types (minimum set): envelope_created, envelope_baked, invitation_sent, invitation_resent, reminder_sent, reminder_cap_reached, token_revoked, document_viewed, field_completed, evidence_uploaded, evidence_accessed, consent_given, signed, declined, voided, expired, completed, copies_distributed, webhook_delivered, webhook_dead_lettered.

The hash chain (each row includes the previous row's hash) means any retroactive tampering breaks the chain visibly. Cheap to implement, significant evidential value.

13. users

Internal staff only. Signers never get accounts.

```
users (
  id          uuid PK,
  email       text UNIQUE,
  name        text,
  role        text CHECK (role IN ('admin','operator','viewer')),
  is_active   boolean,
  created_at  timestampz
)
```

```
user_entity_access (  
  user_id    uuid FK -> users,  
  entity_id  uuid FK -> entities,  
  PRIMARY KEY (user_id, entity_id)  
)
```

Auth via Google Identity (you're in the Google stack anyway). Entity access controls which sending companies a staff member can create envelopes for.

Reminders

Two paths, one send mechanism (the entity's 'reminder' email template):

Automated. Set per envelope at creation via `reminder_frequency` (none / daily / every 3 days / weekly). A scheduled job scans `in_progress` envelopes and sends to parties who are currently actionable and have not completed. Rules:

- Sequential mode: only the party whose turn it is. Never remind a party who cannot yet act.
- Cap: `max_auto_reminders` per party (default 5), then automated reminders stop for that party. Log `reminder_cap_reached`.
- Suppression: no reminders to completed or declined parties, none on envelopes in any terminal state, none within 24h of the original invitation, and automated reminders never fire twice within their own interval (job is idempotent against the last `reminder_sent` event per party).
- Hard bounce on a party's address pauses all sends to that party and flags the envelope for staff attention. Continuing to send into a bounce burns the sending domain's reputation.

Manual. A "send reminder now" action per party in the staff UI, available whenever that party is actionable. Does not count against the automated cap. Logged as `reminder_sent` with `actor = staff:<user_id>`.

Webhooks (external system integration)

Fires on envelope status transitions so the consuming system can drive a red/amber/green client view: completion-only would make amber and red invisible.

```
webhook_endpoints (  

```

```

    id            uuid PK,
    entity_id     uuid FK -> entities NULL, -- NULL = global endpoint, all entities
    url           text NOT NULL,             -- HTTPS only
    secret        text NOT NULL,             -- HMAC-SHA256 signing key
    events        text[] NOT NULL,          -- which transitions to deliver
    is_active     boolean DEFAULT true,
    created_at    timestampz

)

webhook_deliveries (
    id            uuid PK,
    endpoint_id   uuid FK -> webhook_endpoints,
    envelope_id   uuid FK -> envelopes,
    event_type    text NOT NULL,
    payload       jsonb NOT NULL,
    attempt_count int DEFAULT 0,
    last_status   int,                      -- HTTP status of last attempt
    delivered_at  timestampz,
    next_retry_at timestampz,
    created_at    timestampz

)

```

Payload shape (stable contract, version it from day one):

```

{
  "version": 1,
  "event": "envelope.completed",
  "envelope_id": "...",
  "external_client_ref": "...",
  "entity_slug": "acme",
  "status": "completed",
  "suggested_light": "green",
  "parties": [
    { "role": "company", "status": "completed" },
    { "role": "agency", "status": "completed" },
    { "role": "supplier", "status": "completed" }
  ],
  "occurred_at": "..."
}

```

Rules:

- Events emitted: envelope.sent, envelope.in_progress, envelope.party_completed, envelope.completed, envelope.declined, envelope.voided, envelope.expired.
- suggested_light mapping the consumer can use or ignore: green = completed; amber =

sent / in_progress / party_completed; red = declined / voided / expired.

- Every request signed: X-Signature header = HMAC-SHA256(secret, body). The consuming system verifies before trusting anything.
- Delivery is at-least-once with retries (backoff: 1m, 5m, 30m, 2h, 12h, then dead-letter and flag for staff). Consumer must be idempotent on (envelope_id, event, occurred_at).
- Since the receiving side of your existing software is unspecified, this payload IS the integration contract: build the receiver to it later, and nothing here needs to change.

File format and size constraints

Enforced server-side in all cases. Declared MIME and file extension are client-controlled and untrustworthy: validate magic bytes, then re-check the parsed structure (a real PDF parser, a real image decoder). Reject on any mismatch between extension, declared MIME, and actual content.

Evidence uploads (proof documents)

Constraint	Value
Accepted formats	PDF, JPEG, PNG, HEIC (converted to JPEG on ingest)
Max size per file	10 MB
Files per requirement	1 (multi-page proofs go in as a single PDF)
PDF page limit	20 pages
Min image dimension	1000 px on the long edge (legibility floor)
Rejected outright	Password-protected/encrypted PDFs, PDFs containing JavaScript or embedded files, zero-byte files, animated formats

Rules:

- HEIC is converted to JPEG for display, but the ORIGINAL upload is what gets stored and

hashed. Never hash a derivative and call it the evidence.

- Do not strip EXIF from evidence images. Capture metadata (device, timestamp, GPS if present) has evidential value. It lives inside an encrypted bucket anyway, so the privacy exposure is controlled by access, not by stripping.
- Malware scan on ingest (ClamAV or GCS-native scanning) before the file is accepted into the evidence bucket. Quarantine on hit, log the event.
- On rejection, the party sees a specific reason ('file is password protected', 'image too small to read'), not a generic failure.

Source contract (fetched from the template link)

Constraint	Value
Accepted sources	PDF, DOCX, Google Docs URL (exported to PDF via Drive API)
Max size	20 MB
Max pages	100
Normalisation	Everything is rendered to flattened PDF at snapshot time; that flattened PDF is what gets hashed and versioned
Rejected outright	Encrypted PDFs, PDFs with JavaScript, fillable form fields (flatten or reject: flatten), external references that fail to resolve

The snapshot must be self-contained: fonts embedded, images embedded, no external fetches at render time. A document that renders differently on two machines cannot be the thing three parties signed.

Cover assets (front/back cover library)

Constraint	Value
Accepted formats	Single-page PDF or PNG
Max size	5 MB
Page size	Must match the contract page size (A4); validated at asset upload, re-validated at bake

Resolution (PNG)	300 DPI at A4 (2480 x 3508 px) minimum
Not accepted	SVG (script injection surface), multi-page files

Signature images (method = 'uploaded' or 'drawn')

Constraint	Value
Accepted formats	PNG (drawn signatures are rendered to PNG server-side)
Max size	2 MB
Processing	Re-encoded server-side; the re-encoded PNG is what is embedded in the completed document

Where these live in the schema

Defaults are system-wide constants; `evidence_requirements.accepted_mimes` and `max_size_bytes` can tighten (never loosen) them per requirement. Example seed for the client role's four requirements:

requirement_key	accepted formats	max size	verify_method
proof_of_address	PDF, JPEG, PNG, HEIC	10 MB	upload
proof_of_directors	PDF, JPEG, PNG, HEIC	10 MB	upload (or companies_house_api)
vat_certificate	PDF, JPEG, PNG, HEIC	10 MB	upload (or hmrc_vat_api)
incorporation_cert	PDF, JPEG, PNG, HEIC	10 MB	upload (or companies_house_api)

Bake pipeline (the critical async job)

1. Fetch `source_url`. Render to PDF. Hash. Create/reuse `template_version`.

2. Validate: covers exist and match page size, all three parties assigned, entity domain_verified = true, all role fields have positions.
3. Assemble: front cover + snapshot + back cover. Overlay field placeholders.
4. Render final PDF. Compute baked_hash. Write documents row (kind = baked).
5. Stamp entity legal details (name, company number, registered office) from the entities row into the document footer or execution block.
6. Set envelope to ready. Log envelope_baked with the hash in metadata.

Any failure returns the envelope to draft with a human-readable error. Never leave an envelope stuck in baking.

Signing flow per party (sequential mode)

1. Party 1's turn: invitation rendered from the entity's 'invitation' email template, entity signature appended, sent from the entity's verified from_address. Log invitation_sent with template id and rendered-content hash.
2. Party opens link: token validated, document_viewed logged with IP/UA.
3. Party completes only their role's fields. Each save logs field_completed.
4. Evidence gate (client role only): signing stays locked until every is_required requirement is satisfied, i.e. each has either an evidence_file in 'uploaded'/'accepted' status or a verification_result of 'verified'. The UI shows the checklist (proof of address, proof of directors, VAT certificate, incorporation certificate) with per-item status. Each upload logs evidence_uploaded with the requirement_key in metadata.
5. Consent checkbox (verbatim text stored), then sign. Signature row written with document_hash = baked_hash. Token burned.
6. Party status -> completed. Next party's invitation fires automatically.
7. After party 3: completion job renders 'completed' doc + 'certificate', distributes to all three parties, logs copies_distributed, envelope -> completed.